

Coding Cheatsheet: Control Structures and String Handling

Estimated duration: 10 minutes

This reading provides a reference list of code that you'll encounter as you explore control structures and string handling. Understanding how to apply this code will help you write and debug your first Java programs.

- Using Conditional Statements
- Introduction to Loops in Java
- Working with Strings in Java
- Using Packages and Imports
- Implementing Functions andMethods

Keep this summary reading available as a reference as you progress through your course, and refer to this reading as you begin coding with Java after this course!

Using Conditional Statements

if statement

The `if` statement checks a condition. If the condition is `true`, it executes the code inside the block. If the condition is `false`, the program skips the `if` block.

Description	Example
A Java class named <code>Main</code> with a <code>main</code> method. The <code>main</code> method is the entry point of the program.	<pre>public class Main {</pre>
The <code>main</code> method is declared using <code>public static void main(String[] args)</code> . This method is required for execution in Java programs.	<pre>public static void main(String[] args) {</pre>
A variable <code>number</code> of type <code>int</code> is declared and initialized with the value <code>10</code> .	<pre>int number = 10;</pre>
The <code>if</code> statement checks if <code>number</code> is greater than 5. If true, it prints "The number is greater than 5."	<pre>if (number > 5) { System.out.println("The number is greater than 5."); }</pre>
Closing curly braces to end the <code>main</code> method and class definition.	<pre> } }</pre>

Explanation: Since `number` is `10`, which is greater than `5`, the condition evaluates to `true`, and the program prints "The number is greater than 5."

if-else statement

The if-else statement gives an alternative if the condition is false.

Description	Example
A Java class named Main with a main method. The main method is the entry point of the program.	<pre>public class Main {</pre>
The main method is declared using public static void main(String[] args). This method is required for execution in Java programs.	<pre> public static void main(String[] args) {</pre>
A variable number of type int is declared and initialized with the value 3.	<pre> int number = 3;</pre>
The if statement checks if number is greater than 5. If true, it prints "The number is greater than 5."	<pre> if (number > 5) { System.out.println("The number is greater than 5."); }</pre>
The else block executes when the if condition is false, printing "The number is not greater than 5."	<pre> else { System.out.println("The number is not greater than 5."); }</pre>
Closing curly braces to end the main method and class definition.	<pre> } }</pre>

else if statement

You can check multiple conditions using else if.

Description	Example
A Java class named Main with a main method. The main method is the entry point of the program.	<pre>public class Main {</pre>

Description	Example
The <code>main</code> method is declared using <code>public static void main(String[] args)</code> . This method is required for execution in Java programs.	<pre>public static void main(String[] args) {</pre>
A variable <code>number</code> of type <code>int</code> is declared and initialized with the value 5.	<pre>int number = 5;</pre>
The <code>if</code> statement checks if <code>number</code> is greater than 5. If true, it prints "The number is greater than 5."	<pre>if (number > 5) { System.out.println("The number is greater than 5."); }</pre>
The <code>else if</code> statement checks if <code>number</code> is exactly 5. If true, it prints "The number is equal to 5."	<pre>else if (number == 5) { System.out.println("The number is equal to 5."); }</pre>
The <code>else</code> block executes when none of the above conditions are met, printing "The number is less than 5."	<pre>else { System.out.println("The number is less than 5."); }</pre>
Closing curly braces to end the <code>main</code> method and class definition.	<pre>} }</pre>

Explanation: Since `number` is exactly 5, the program prints "The number is equal to 5."

switch statement

A switch statement checks a single variable against multiple values.

Description	Example
A Java class named <code>Main</code> with a <code>main</code> method. The <code>main</code> method is the entry point of the program.	<pre>public class Main {</pre>
The <code>main</code> method is declared using <code>public static void main(String[] args)</code> . This method is required for execution in Java programs.	<pre>public static void main(String[] args) {</pre>
A variable <code>day</code> of type <code>int</code> is declared and initialized with the value <code>3</code> .	<pre>int day = 3;</pre>
The <code>switch</code> statement checks the value of <code>day</code> and compares it against the case labels. If <code>day</code> is <code>3</code> , it prints "Wednesday".	<pre>switch (day) { case 1: System.out.println("Monday"); break; case 2: System.out.println("Tuesday"); break; case 3: System.out.println("Wednesday"); break; case 4: System.out.println("Thursday"); break; case 5: System.out.println("Friday"); break; default: System.out.println("Weekend"); }</pre>
Closing curly braces to end the <code>main</code> method and class definition.	<pre> } }</pre>

default case in a switch statement

When using a `switch` statement, it's a good practice to specify a default case. This case runs if none of the specified cases match, acting as a fallback option.

Description	Example
A switch statement checks the value of a variable <code>color</code> .	<pre>switch (color) {</pre>

Description	Example
A case checks if color is "red", printing "Color is red."	<pre>case "red": System.out.println("Color is red."); break;</pre>
A case checks if color is "blue", printing "Color is blue."	<pre>case "blue": System.out.println("Color is blue."); break;</pre>
The default case prints "Unknown color." if color doesn't match "red" or "blue".	<pre>default: System.out.println("Unknown color."); }</pre>

Nested Conditional Statements

You can place conditional statements within each other to create more complex decisions. The process of placing conditional statements within other conditional statements is called nesting.

Description	Example
A Java class named Main with a main method. The main method is the entry point of the program.	<pre>public class Main {</pre>
The main method is declared using public static void main(String[] args). This method is required for execution in Java programs.	<pre>public static void main(String[] args) {</pre>
A variable age of type int is declared and initialized with the value 20.	<pre>int age = 20;</pre>

Description	Example
The if statement checks if age is greater than or equal to 18. If true, it prints "You are an adult."	<pre>if (age >= 18) { System.out.println("You are an adult."); }</pre>
Another if statement checks if age is greater than or equal to 65, printing "You are a senior citizen." if true.	<pre>if (age >= 65) { System.out.println("You are a senior citizen."); }</pre>
The else block executes if age is less than 18, printing "You are a minor."	<pre>else { System.out.println("You are a minor."); }</pre>
Closing curly braces to end the main method and class definition.	<pre>} }</pre>

Introduction to Loops in Java

for Loop

The for loop is used when the number of iterations is known beforehand. It consists of three parts:

- Initialization: This sets a counter variable.
- Condition: This checks if the loop should continue executing.
- Increment/Decrement: This updates the counter variable after each iteration.

Description	Example
A Java class named ForLoopExample with a main method. The main method is the entry point of the program.	<pre>public class ForLoopExample {</pre>
The main method is declared using public static void main(String[] args). This method is required for execution in Java programs.	<pre>public static void main(String[] args) {</pre>

Description	Example
A for loop is initialized with <code>int i = 1</code> , which starts the counter at 1. The counter variable <code>i</code> is incremented by <code>i++</code> after each iteration.	<code>for (int i = 1; i <= 5; i++) {</code>
The loop checks if <code>i <= 5</code> , and if true, it prints the value of <code>i</code> .	<code>System.out.println(i);</code>
Close the for loop.	<code>}</code>
Close the main method.	<code>}</code>
Close the <code>ForLoopExample</code> class.	<code>}</code>

while Loop

The while loop is used when the number of iterations is not known in advance. It continues executing as long as the specified condition remains true.

Description	Example
A Java class named <code>WhileLoopExample</code> with a main method. The main method is the entry point of the program.	<code>public class WhileLoopExample {</code>
The main method is declared using <code>public static void main(String[] args)</code> . This method is required for execution in Java programs.	<code>public static void main(String[] args) {</code>

Description	Example
A variable <code>i</code> is initialized to 1.	<pre>int i = 1;</pre>
The while loop continues as long as <code>i <= 5</code> .	<pre>while (i <= 5) {</pre>
Inside the loop, the value of <code>i</code> is printed, then incremented by <code>i++</code> .	<pre> System.out.println(i); i++; }</pre>
The <code>main</code> method and class are closed with curly braces.	<pre>}</pre>

do-while Loop

The do-while loop is similar to the while loop but guarantees that the code block executes at least once before checking the condition.

Description	Example
A Java class named <code>DowhileLoopExample</code> with a <code>main</code> method. The <code>main</code> method is the entry point of the program.	<pre>public class DowhileLoopExample {</pre>
The <code>main</code> method is declared using <code>public static void main(String[] args)</code> . This method is required for execution in Java programs.	<pre> public static void main(String[] args) {</pre>
A variable <code>i</code> is initialized to 1.	<pre> int i = 1;</pre>

Description	Example
The do-while loop starts and executes at least once before checking if <code>i <= 5</code> .	<pre>do {</pre>
Inside the loop, the value of <code>i</code> is printed, then incremented by <code>i++</code> .	<pre> System.out.println(i); i++;</pre>
The condition <code>i <= 5</code> is checked after each iteration.	<pre>} while (i <= 5);</pre>
The <code>main</code> method and class are closed with curly braces.	<pre>} }</pre>

Nested loops

You can also use loops inside other loops, known as nested loops. This is useful for working with multi-dimensional data structures, like arrays or matrices.

Description	Example
A Java class named <code>NestedLoopsExample</code> with a <code>main</code> method. The <code>main</code> method is the entry point of the program.	<pre>public class NestedLoopsExample {</pre>
The <code>main</code> method is declared using <code>public static void main(String[] args)</code> . This method is required for execution in Java programs.	<pre> public static void main(String[] args) {</pre>
The outer loop controls the rows, running 10 times.	<pre> for (int i = 1; i <= 10; i++) {</pre>

Description	Example
The inner loop controls the columns, also running 10 times for each row.	<pre>for (int j = 1; j <= 10; j++) {</pre>
The product of <code>i * j</code> is printed for each combination of rows and columns.	<pre>System.out.print(i * j + "\t");}</pre>
After the inner loop, a newline is printed to separate the rows.	<pre>System.out.println();}</pre>
The <code>main</code> method and class are closed with curly braces.	<pre>} }</pre>

break statement

The `break` statement is used to terminate a loop immediately, regardless of the loop's condition. This can be useful when you want to exit a loop based on a specific condition that may occur during its execution.

Description	Example
A Java class named <code>BreakExample</code> with a <code>main</code> method. The <code>main</code> method is the entry point of the program.	<pre>public class BreakExample {</pre>
The <code>main</code> method is declared using <code>public static void main(String[] args)</code> . This method is required for execution in Java programs.	<pre>public static void main(String[] args) {</pre>
An array <code>numbers</code> is declared and initialized.	<pre>int[] numbers = {1, 3, 5, 7, 9, 2, 4};</pre>

Description	Example
The loop iterates through the array, checking if any number is greater than 5.	<pre>for (int num : numbers) {</pre>
When a number greater than 5 is found, it is printed and the loop exits with the break statement.	<pre> if (num > 5) { System.out.println(num); break; }</pre>
The main method and class are closed with curly braces.	<pre>}</pre> <pre>}</pre> <pre>}</pre>

continue statement

The continue statement is used to skip the current iteration of a loop and move to the next iteration. It's useful when you want to skip certain conditions but continue executing the rest of the loop.

Description	Example
A Java class named ContinueExample with a main method. The main method is the entry point of the program.	<pre>public class ContinueExample {</pre>
The main method is declared using public static void main(String[] args). This method is required for execution in Java programs.	<pre> public static void main(String[] args) {</pre>
The loop iterates through the numbers from 1 to 10.	<pre> for (int i = 1; i <= 10; i++) {</pre>
When i == 5, the continue statement is executed, skipping the System.out.println(i); statement for that iteration.	<pre> if (i == 5) { continue; }</pre>

Description	Example
The value of <code>i</code> is printed for all numbers except 5.	<pre>System.out.println(i);</pre>
The <code>main</code> method and class are closed with curly braces.	<pre>} } }</pre>

Working with Strings in Java

Creating strings

You can create a string in Java in two main ways:

Using string literals: This means writing the string directly inside double quotes.

Description	Example
Create a string using string literal.	<pre>String greeting = "Hello, World!";</pre>

In this example, we created a string called `greeting` that contains "Hello, World!".

Using the new Keyword: This method involves using the `new` keyword to create a string object.

Description	Example
Create a string using the <code>new</code> keyword.	<pre>String message = new String("Hello, World!");</pre>

Although this works, it's more common to use string literals because it's simpler.

String length

To find out how many characters are in a string, you can use the `length()` method. This method tells you the total number of characters in the string.

Description	Example
Create a string and use length() to get the number of characters.	<pre>String text = "Java Programming";</pre>
Use the length() method to find the string length.	<pre>int length = text.length();</pre>
Print the length of the string.	<pre>System.out.println("Length of the string: " + length); // Output: 16</pre>

Here, we created a string called text and then checked its length. The output tells us that there are 16 characters in "Java Programming".

Accessing characters

If you want to look at individual characters in a string, you can use the charAt() method. This method allows you to get a character at a specific position in the string.

Description	Example
Create a string and access a character using charAt().	<pre>String word = "Java";</pre>
Access the first character of the string.	<pre>char firstChar = word.charAt(0);</pre>
Print the first character of the string.	<pre>System.out.println("First character: " + firstChar); // Output: J</pre>

In this example, we accessed the first character of the string "Java". Remember that counting starts at 0, so charAt(0) gives us 'J'.

Concatenating strings

Sometimes you might want to combine two or more strings together. You can do this easily using the + operator or the concat() method.

Description	Example
Combine two strings using the + operator.	<pre>String firstName = "John";</pre>
Combine two strings using the + operator.	<pre>String lastName = "Doe";</pre>
Concatenate first and last names using the + operator.	<pre>String fullName = firstName + " " + lastName;</pre>
Print the full name.	<pre>System.out.println("Full Name: " + fullName); // Output: John Doe</pre>

Here, we combined firstName and lastName using the + operator and added a space between them.

You can also use the concat() method:

Description	Example
Combine strings using the concat() method.	<pre>String anotherFullName = firstName.concat(" ").concat(lastName);</pre>
Print the concatenated string.	<pre>System.out.println("Another Full Name: " + anotherFullName); // Output: John Doe</pre>

String comparison

When you want to check if two strings are the same, use the equals() method. This checks if both strings have identical content.

Description	Example
Create three strings to compare.	<pre>String str1 = "Hello";</pre>

Description	Example
Create another string to compare.	<pre>String str2 = "Hello";</pre>
Create a third string to compare.	<pre>String str3 = "World";</pre>
Check if str1 is equal to str2.	<pre>boolean isEqual = str1.equals(str2);</pre>
Print comparison result.	<pre>System.out.println("str1 equals str2: " + isEqual); // Output: true</pre>
Check if str1 is equal to str3.	<pre>boolean isNotEqual = str1.equals(str3);</pre>
Print comparison result.	<pre>System.out.println("str1 equals str3: " + isNotEqual); // Output: false</pre>

In this example, isEqual returns true because both strings are "Hello". However, isNotEqual returns false since "Hello" and "World" are different.

String immutability

One important thing to know about strings in Java is that they are immutable. This means that once a string is created, it cannot be changed. If you try to change it, you will actually create a new string instead.

Description	Example
Create an original string.	<pre>String original = "Hello";</pre>

Description	Example
Add to the string (creates a new string).	<pre>original = original + " World";</pre>
Print the new string.	<pre>System.out.println(original); // Output: Hello World</pre>

In this case, we added "World" to original, but instead of changing the original string, we created a new string that combines both parts.

Finding substrings

You may want to get a smaller part of a string. You can do this using the `substring()` method, which allows you to specify where to start and where to stop.

Description	Example
Create a string and extract a substring.	<pre>String phrase = "Java Programming";</pre>
Extract a substring from the string.	<pre>String sub = phrase.substring(5, 16);</pre>
Print the extracted substring.	<pre>System.out.println("Substring: " + sub); // Output: Programming</pre>

In this example, we started at index 5 and went up to (but did not include) index 16 to extract "Programming".

String methods

Java has many built-in methods for strings that help you manipulate and process them. Here are some useful ones:

toUpperCase(): This method converts all letters in a string to uppercase.

Description	Example
Create a string.	<pre>String text = "hello";</pre>
Convert the string to uppercase.	<pre>System.out.println(text.toUpperCase()); // Output: HELLO</pre>

toLowerCase(): This converts all letters in a string to lowercase.

Description	Example
Create a string.	<pre>String text = "WORLD";</pre>
Convert the string to lowercase.	<pre>System.out.println(text.toLowerCase()); // Output: world</pre>

trim(): This method removes any extra spaces at the beginning or end of a string.

Description	Example
Create a string with extra spaces and trim it.	<pre>String textWithSpaces = " Hello ";</pre>
Remove spaces from the string.	<pre>System.out.println(textWithSpaces.trim()); // Output: Hello</pre>

replace(): If you want to change all instances of one character or substring to another, use this method.

Description	Example
Create a sentence and replace a word.	<pre>String sentence = "I like cats.";</pre>

Description	Example
Replace a word in the sentence.	<pre>String newSentence = sentence.replace("cats", "dogs");</pre>
Print the new sentence.	<pre>System.out.println(newSentence); // Output: I like dogs.</pre>

Splitting strings

You can break a string into smaller pieces using the `split()` method. This is useful when you have data separated by commas or spaces.

Description	Example
Create a CSV string and split it.	<pre>String csv = "apple,banana,cherry";</pre>
Split the string at each comma.	<pre>String[] fruits = csv.split(",");</pre>
Print each fruit in the array.	<pre>for (String fruit : fruits) { System.out.println(fruit); }</pre>
Output:	<pre>apple banana cherry</pre>

Joining strings

If you have an array of strings and want to combine them back into one single string, you can use the `String.join()` method.

Description	Example
Create an array of strings.	<pre>String[] colors = {"Red", "Green", "Blue"};</pre>
Join the strings with a separator.	<pre>String joinedColors = String.join(", ", colors);</pre>
Print the joined string.	<pre>System.out.println(joinedColors); // Output: Red, Green, Blue</pre>

Using Packages and Imports

Creating a package

To create a package, you simply declare it at the top of your Java source file using the `package` keyword followed by the package name. For example:

Description	Example
Declare a package at the top of the file.	<pre>package com.example.myapp;</pre>

In this example, `com.example.myapp` is the name of the package. It's common practice to use a reverse domain name as the package name to ensure uniqueness.

Description	Example
Define a class inside the package.	<pre>public class MyClass { // Class code here }</pre>

Creating and using a package

Description	Example
Define a class inside the shapes package.	<pre>package shapes;</pre>

Description	Example
Create the Circle class with a constructor and a method.	<pre>public class Circle { private double radius; public Circle(double radius) { this.radius = radius; } public double area() { return Math.PI * radius * radius; } }</pre>

To use this class in another Java file, you need to import it.

Importing classes

To import a specific class from a package, use the following syntax:

Description	Example
Import a specific class from a package.	<pre>import package_name.ClassName;</pre>

Importing all classes from a package

You can also import all classes from a package using an asterisk (*).

Description	Example
Import all classes from the shapes package.	<pre>import shapes.*;</pre>

This imports all classes in the shapes package, allowing you to use any class without needing to import them individually.

Implementing Functions and Methods

Function structure

Description	Example
The structure of a function in Java.	<pre>returnType functionName(parameter1Type parameter1, parameter2Type parameter2) { // code to be executed return value; // optional }</pre>

Description	Example

Example of a Simple Function

Let's create a simple function that adds two numbers:

Description	Example
Create a function that adds two numbers.	<pre>public static int add(int a, int b) { return a + b; }</pre>
Call the add function in the main method and print the result.	<pre>int sum = add(5, 3); System.out.println("The sum is: " + sum);</pre>

Example of a simple method

Let's create a method within a class:

Description	Example
Define a multiply method inside the Calculator class.	<pre>public class Calculator {</pre>
The multiply method takes two integers and returns their product.	<pre> public int multiply(int x, int y) {</pre>
Close the multiply method.	<pre> return x * y; }</pre>
Define the main method, which is the program's entry point.	<pre> public static void main(String[] args) {</pre>

Description	Example
Create an instance of the Calculator class in the main method.	<pre>Calculator calc = new Calculator();</pre>
Call the multiply method with 4 and 5 and store the result.	<pre>int product = calc.multiply(4, 5);</pre>
Print the result to the screen.	<pre>System.out.println("The product is: " + product);</pre>
Close the main method and class.	<pre>} }</pre>

Parameters and arguments

Parameters are the inputs to functions or methods, while arguments are the values passed when calling these functions or methods.

Description	Example
Define a method with multiple parameters.	<pre>public void greet(String name, int age) { System.out.println("Hello " + name + ", you are " + age + " years old."); }</pre>
Call the greet method with arguments in the main method.	<pre>Greeting greeting = new Greeting(); greeting.greet("Alice", 30);</pre>

Return values

Functions and methods can return values or perform actions without returning anything.

Description	Example
Define a method that returns a value.	<pre>public double area(double length, double width) { return length * width; }</pre>
Call the area method and print the returned value.	<pre>Rectangle rect = new Rectangle(); double area = rect.area(4.5, 3.0); System.out.println("The area of the rectangle is: " + area);</pre>

Overloading methods

Java allows defining multiple methods with the same name but different parameters. This is known as method overloading.

Description	Example
Define the Display class.	<pre>public class Display {</pre>
Define an overloaded method that takes an int.	<pre>public void show(int number) {</pre>
Print the number.	<pre>System.out.println("Number: " + number);</pre>
Close the first method.	<pre>}</pre>
Define an overloaded method that takes a String.	<pre>public void show(String text) {</pre>

Description	Example
Print the text.	<code>System.out.println("Text: " + text);</code>
Close the second method.	<code>}</code>
Define the main method to call the overloaded methods.	<code>public static void main(String[] args) {</code>
Create a Display object.	<code>Display display = new Display();</code>
Call show(int) and show(String).	<code>display.show(10);</code>
<code>display.show("Hello World");</code>	
Close the main method.	<code>} }</code>

Scope of identifiers

The scope of an identifier refers to the part of the program where the identifier can be accessed.

Description	Example
Local Scope: Identifiers are accessible only within the method or block.	<code>int x = 10; // x is local to this block</code>

Description	Example
Instance Scope: Variables are accessible by all methods in the class.	<pre>private int x; // x is accessible by all methods</pre>
Static Scope: Static variables belong to the class and are accessible throughout the class.	<pre>private static int count;</pre>

Void methods

A void method does not return a value.

Description	Example
Define a void method that prints a message.	<pre>public void printMessage() { System.out.println("Hello, World!"); }</pre>

Empty parameter lists

An empty parameter list means the method does not take any parameters.

Description	Example
Define a method with an empty parameter list.	<pre>public void show() { System.out.println("No parameters here."); }</pre>

Author(s)

[Ramanujam Srinivasan](#)

[Lavanya Thiruvalli Sunderarajan](#)



Skills Network